

Final Review Report

***“Driving Strategies for Omnidirectional Mobile Robots with Offset Differential Wheels”
(Badia Torres, Perez Gracia, and Domenech-Mestres, 2024)***

**Comparative Analysis of Structural and Parametric Instability in Trajectory-Tracking Control:
MOBY (Offset-Differential) vs. F1TENTH (Ackermann)**

I PAO 2026 - 2027

Facultad de Ingeniería Mecánica y Ciencias de la Producción
Escuela Superior Politécnica del Litoral
Guayaquil – Ecuador

Carolina Isabella Sánchez Cevallos,
carisanc@espol.edu.ec

Jhony David Choez Lopez,
jdchoez@espol.edu.ec

1. ABSTRACT

This report is a critical review and independent extension of Badia Torres, Perez Gracia, and Domenech-Mestres (2024), “Driving Strategies for Omnidirectional Mobile Robots with Offset Differential Wheels,” which characterizes a structural trajectory-tracking instability in the MOBY offset-differential robot: a stable “pure push” equilibrium and an unstable “pure pull” equilibrium, governed by the robot’s fixed offset d_1 . Rather than reviewing the paper in isolation, this project places it against a mechanistically different but mathematically related instability found in Ackermann-steered vehicles under pure pursuit control (Coulter, 1992; Evans et al., 2024), using the F1TENTH platform as the experimental counterpart. The central question this comparison must answer is: since only F1TENTH could be physically/gym-simulated, what exactly is being compared, and what is learned or improved?

The answer developed in this report has three parts. First, an independent numerical reproduction of MOBY’s closed-form push/pull dynamics (Eq. 20–25 of the source paper) was built from scratch in Python, giving the offset-differential case its own runnable simulation, confirming the reported instability, validating a compensation law against the paper’s own documented behavior for its aggressiveness parameter, and deriving a closed-form curvature-feasibility limit ($R = d_1$) analogous to the paper’s design rule. Second, a standalone kinematic pure pursuit simulation was built for F1TENTH and exercised on a path matching the curvature profile of MOBY’s own test trajectory, testing the effect of path curvature on tracking error before any ROS2 infrastructure was needed. Third, the same experiment was repeated on the real F1TENTH ROS2/gym stack, a from-scratch pipeline of SLAM (slam_toolbox), localization (tuned nav2 AMCL), path planning (custom minimum-curvature raceline with a friction-circle velocity profile), and a from-scratch pure pursuit controller, driven around a real 400-waypoint São Paulo circuit at three look-ahead settings, with tracking error logged and correlated against the raceline’s own curvature and velocity columns.

The results show that MOBY’s instability is fixed by geometry and can only be handled by an active, real-time compensation algorithm, while F1TENTH’s tracking error is continuously tunable through a single controller parameter, the same look-ahead root cause, two structurally different remedies. A further, unanticipated finding from the real F1TENTH data is that tracking error is worst not at the tightest curvature (the hairpin), but at a moderate curvature still taken at high commanded speed, revealing a speed-curvature interaction that the closed-form MOBY model and the constant-speed kinematic F1TENTH simulation both omit, a concrete, quantified limitation of the simplified models used in this project and a natural bridge to the MPC-based control planned as future work.

2. INTRODUCTION

Trajectory-tracking instability is a well-documented phenomenon in wheeled mobile robotics: whenever a controller regulates a point offset from the robot’s own actuation frame, the internal orientation of the chassis relative to the direction of travel can itself become an unstable state variable (Yun and Yamamoto, 1997). Badia Torres, Perez Gracia, and Domenech-Mestres (2024) study this phenomenon in detail for MOBY, an offset-differential omnidirectional robot developed and patented at the Polytechnic University of Catalonia, and propose driving strategies, including a velocity-mirroring compensation algorithm, that allow the robot to operate productively even inside its structurally unstable “pull” region, which is advantageous on rough or uneven terrain.

This report is structured as a comparative study, contrasting MOBY’s instability with the geometrically related, but mechanistically different, look-ahead instability found in Ackermann-steered vehicles under pure pursuit control (Coulter, 1992), using the F1TENTH 1:10-scale autonomous racing platform (O’Kelly et al., 2020) as the counterpart. This comparative framing is not cosmetic: it requires building an entire secondary software stack, covering theory, standalone simulation, and a full ROS2 navigation pipeline, to give the comparison real content, addressed in detail throughout this report.

The remainder of this report is organized as follows. Section 3 states the problem this project addresses, including the specific gaps this comparison must close. Section 4 summarizes the MOBY paper. Section 5 places it in the context of the broader look-ahead-control literature. Section 6 is the technical critique proper (strengths, weaknesses, organization/style). Section 7 fixes the notation used throughout, to keep the two source literatures from being read against each other ambiguously. Section 8 details the methodology: the kinematic models, the pure pursuit

derivation, and the design of three independent verification tracks. Section 9 presents the computations, simulations, and real-world example results. Section 10 discusses what these results mean, including their limitations. Sections 11–12 conclude and list references, and the Annexes contain repository structure, code organization, validation evidence, and supporting figures that are better shown as diagrams or screenshots than as in-body code listings.

3. PROBLEM STATEMENT

Both MOBY and FITENTH exhibit trajectory-tracking instabilities that, left unaddressed, degrade tracking accuracy, but the origin of each instability differs fundamentally. In MOBY, the pure-pull equilibrium is unstable because of the robot's offset-differential geometry: if uncorrected, the chassis spontaneously rotates away from the pull direction toward the stable push direction, producing trajectory deviations, torque peaks, increased friction demands, and loss of odometry precision (Badia Torres et al., 2024). In FITENTH, under pure pursuit control, tracking behavior depends continuously on the look-ahead distance chosen relative to the vehicle's kinematic constraints: a short look-ahead improves accuracy but increases oscillation, while a long look-ahead is smoother but cuts corners (Coulter, 1992).

Although these two problems arise from different mechanisms, one embedded in a robot's fixed physical architecture, the other in a tunable control parameter, both can be read through the same look-ahead stability framework established by Yun and Yamamoto (1997). This is the theoretical anchor of the comparison. A comparison built this way carries a risk of asymmetry: FITENTH can be exercised on a full ROS2/gym stack, but MOBY has no physical offset-differential platform available, and reviewing its paper alone would leave the comparison one-sided, theory on one side, simulation on the other, with no clear answer to what is actually being compared or improved. A second risk is that a single fixed test path understates how each instability behaves, since both the MOBY paper's own design rule (avoid turning radii tighter than d_1) and the pure pursuit look-ahead trade-off are inherently curvature-dependent. A third risk is notational: several symbols are reused across the two source literatures with different meanings — (x, y) , " d ," and " γ " each mean something different in Badia Torres et al. (2024) than in Coulter (1992) — and left undisambiguated, this would make any side-by-side derivation misleading rather than merely imprecise.

This report treats all three risks as first-class requirements, not afterthoughts. Section 7 fixes the notation problem directly. Section 8.1 states explicitly what is compared and why, and Sections 8.4–8.6 describe the three independent verification tracks, a from-scratch MOBY simulation, a standalone FITENTH kinematic simulation on a curvature-matched path, and a real ROS2/gym pipeline exercised at multiple look-ahead settings on a real, curvature-varying circuit, built specifically to close these gaps.

4. SUMMARY OF THE PAPER

Badia Torres, Perez Gracia, and Domenech-Mestres (2024) study MOBY, an offset-differential omnidirectional mobile robot developed and patented at the Polytechnic University of Catalonia (UPC), currently used in logistics applications. MOBY achieves omnidirectionality with two conventional (non-Mecanum) actuated wheels offset a fixed distance d_1 from a third, independently actuated vertical rotation axis, plus a passive caster wheel for support. Because the wheels are offset rather than centered, the robot inherits an instability structurally similar to that of differential-drive robots controlled with a look-ahead point ahead of the wheel axis (Yun and Yamamoto, 1997): a stable equilibrium in which the wheels trail the robot's center of motion ("pure push"), and an unstable equilibrium in which the wheels lead it ("pure pull").

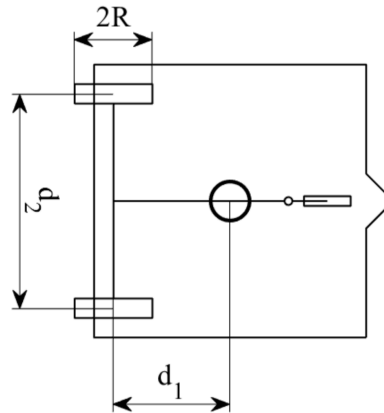


Figure 4.1. The offset-differential structure of MOBY: wheel radius R , wheel separation d_2 , and offset d_1 between the wheel axis and the vertical rotation axis (Badia Torres et al., 2024).

The paper's core contribution is deriving the full kinematic and dynamic model of MOBY, including the previously overlooked passive caster wheel, and using it to explain and then exploit the pull-region instability, rather than simply avoiding it as prior work on similar robots (e.g., "OmniKitty," Jung et al., 1999; "Otbot," Giró et al., 2023) had done. The forward and inverse kinematics are derived via a Jacobian relating wheel/platform velocities to the robot's twist (their Eq. 8–16), and the internal-dynamics stability analysis, built on Yun and Yamamoto's (1997) look-ahead framework, shows that the angle γ between the pure-push direction and the desired velocity evolves as $\dot{\gamma} = -(\sin\gamma/d_1) \cdot c$ (their Eq. 20), with corresponding instantaneous turn radius $1/R_{\text{turn}} = -\sin\gamma/d_1$ (Eq. 21).

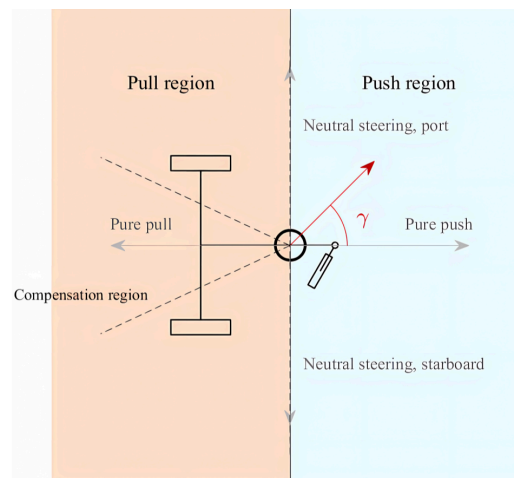


Figure 4.2. Push and pull regions of MOBY as a function of γ (Badia Torres et al., 2024).

Because operating in the pull region is advantageous for rough terrain (the larger wheel diameter and forward-facing orientation better climb obstacles), the authors design a compensation strategy that actively corrects deviations away from a chosen equilibrium by mirroring the commanded velocity vector as a function of the tracking error $\gamma_{err} = \gamma - \gamma_{goal}$, with an aggressiveness parameter k (their Eq. 23–25: $k=0$ is no correction, $k=1$ stops further drift without returning, and $k>1$ actively restores toward γ_{goal}). The paper validates this model and strategy both in MATLAB simulation (reproducing the torque and friction demands of an uncorrected instability event, and the reduced demands under compensation) and experimentally on the physical MOBY robot, including a disturbance-rejection test using simulated impacts on the caster fork.

The paper’s conclusion is that the pull-region instability is a structural, kinematic-origin phenomenon with dynamic consequences, that it can be actively and successfully compensated, and that the caster wheel’s own dynamics can materially affect torque requirements and should be instrumented in high-speed applications. This is the primary literature source for the structural side of the comparison developed in this report.

5. LITERATURE REVIEW

The MOBY paper sits at the intersection of two literatures that this project draws on directly for the F1TENTH side of the comparison.

Look-ahead control stability (shared theoretical root)

Yun and Yamamoto (1997) is the paper Badia Torres et al. build on: it derives the internal-dynamics stability of a wheeled robot tracked via a look-ahead point, using a Lyapunov argument on the angular position of the wheels, and identifies the same push/pull-style equilibrium structure that MOBY inherits. This is the theoretical anchor that lets this report treat MOBY and F1TENTH as two instances of one underlying problem rather than two unrelated case studies.

Pure pursuit and F1TENTH (the comparison’s counterpart literature)

Coulter (1992) is the original technical report deriving the pure pursuit path-tracking algorithm used in this project’s F1TENTH work: a purely geometric method that fits a circular arc between the vehicle and a look-ahead goal point, with a curvature formula $\kappa_{pp} = 2 \cdot y/l^2$ (in Coulter’s own local-frame convention) and an explicit discussion of the responsiveness/stability trade-off in the look-ahead distance, the parametric-instability analogue used throughout this report.

O’Kelly et al. (2020) introduces F1TENTH as an open, low-cost, 1:10-scale autonomous racing platform for research and education, and Evans et al. (2024) is a recent survey unifying the F1TENTH autonomous-racing literature, including the specific steering-angle formulation $\delta = \arctan(L \cdot \sin\phi/l_d)$ used as the practical control law in this project’s simulations and ROS2 controller. Section 8.3 shows, and Section 9 confirms against this project’s own working ROS2 code, that this formula and Coulter’s raw curvature formula are not numerically identical, a literature inconsistency this project quantifies rather than glosses over.

Critical comparison

Reading these two literatures side by side is where this report’s contribution lies: Yun and Yamamoto (1997) and Badia Torres et al. (2024) treat the look-ahead-induced instability as a structural property to be compensated by an active controller because the offending length scale ($d1$) is a physical dimension; Coulter (1992) and Evans et al. (2024) treat the analogous instability in Ackermann vehicles as a tuning problem because the offending length scale (l_d) is a software parameter. Neither literature discusses the other’s mechanism explicitly, this report’s Sections 8–10 build that bridge directly, including with new simulation evidence on both sides.

6. TECHNICAL CRITIQUE

6.1 Major Strengths and Contributions

- Unifies theory, simulation, and physical experiment. Unlike most prior work on offset-differential or similarly-actuated robots, which characterizes only the stable pushing dynamics, the paper explicitly develops and validates strategies for the unstable pull region, backed by MATLAB simulation and hardware tests, a rare combination of completeness at this scope.
- The caster wheel is modeled, not ignored. The authors note explicitly that most publications on this robot class overlook the passive caster's dynamic contribution; their augmented Jacobian and torque analysis (their Section 6) show it can be dynamically significant despite its small mass, and this argument is well supported by their simulation results (their Figures 16–17).
- The compensation strategy is simple, tunable, and clearly derived. The velocity-mirroring correction (Eq. 23–25) reduces to an intuitive geometric operation with a single, well-explained aggressiveness parameter k , and the authors explicitly characterize its qualitative behavior across k , which this project was able to independently validate numerically (Section 9.1) precisely because the description was clear and complete enough to reproduce.
- The design implications are actionable, not just descriptive. Concrete rules (e.g., avoid turning radii smaller than d_1 ; instrument the caster wheel's orientation for high-speed operation) give the paper practical value beyond the specific MOBY platform.

6.2 Weaknesses

- Limited generalization guidance. The stability analysis and compensation law are developed specifically for MOBY's geometry; the paper does not discuss how d_1 , d_2 , or the caster geometry parameters should be chosen at design time to trade off instability severity against other performance goals (e.g., footprint, maneuverability).
- No discussion of sensing requirements for the compensation strategy in realistic conditions. The paper notes that caster-wheel orientation “should be instrumented” for accurate torque control but does not analyze the compensation algorithm's sensitivity to pose/heading estimation error, a gap this report's own F1TENTH experiments (Section 9.4–9.5) show can matter substantially in practice (tracking error is dominated by controller-parameter and speed effects, not by localization error, once localization itself is well-tuned, but this required this project's own instrumentation to observe).
- The MATLAB simulations use idealized friction/ground contact. The paper's dynamic simulations (Section 7) explicitly disable viscous friction at the joints “for clarity,” which is reasonable for isolating the phenomenon of interest but leaves open how much the compensation strategy's benefit would be reduced under more realistic friction and slip, exactly the kind of speed-dependent effect this project's real F1TENTH data (Section 9.5) shows can dominate tracking error in a related system.
- Some notation is locally reused. Within the paper itself, γ is used consistently, but readers cross-referencing it against adjacent literature (e.g., Coulter, 1992, which also uses γ for an unrelated curvature quantity) can be misled if the two are read together without care, the exact issue this project's own notation table (Section 7) was built to prevent from propagating into this report.

6.3 Organization, Style, and Readability

The paper is generally well organized, following a logical progression from kinematics (Section 3) through kinematic behavior and stability (Section 4), driving strategy (Section 5), dynamic modeling (Section 6), experimental validation (Sections 7–8), and conclusions. Equations are numbered and consistently referenced in the text, and each major result (e.g., the push/pull equilibrium, the compensation law) is illustrated with a clear, purpose-built figure.

Two readability issues are worth noting. First, the augmented Jacobian matrices (Eq. 13, 16) are dense and would benefit from an intermediate explanation of which entries correspond to which physical degree of freedom before the full matrix is presented, a reader unfamiliar with the caster-wheel parametrization (d_3 , d_4 , β) can lose track of the geometric meaning. Second, the transition between the kinematic stability analysis (Section 4, single-point/scalar γ dynamics) and the full dynamic model (Section 6, multibody Lagrangian) is somewhat abrupt; a short bridging paragraph explaining how the two levels of description relate would help a reader who wants to reproduce only the simpler (scalar) model, as this project's own MOBY simulation deliberately does (Section 8.4).

7. NOTATION

To avoid exactly this kind of cross-literature ambiguity, Table 7.1 fixes one meaning per symbol for the remainder of this report. Several symbols are reused across the two source literatures (Badia Torres et al., 2024, for MOBY; Coulter, 1992, and Evans et al., 2024, for F1TENTH/pure pursuit) with different meanings; where a symbol had to be renamed relative to its original source, the renaming is noted explicitly in the table and used consistently in every subsequent derivation, figure, and code reference in this report.

Symbol	Model	Meaning
d_1	MOBY	Fixed offset between the actuated-wheel axis and the vertical rotation axis (structural, not tunable).
d_2 , d_3 , d_4 , R , R_c	MOBY	Other fixed structural dimensions: wheel separation, caster fork length/offset, wheel/caster radii.
γ	MOBY	Angle between the pure-push direction and the desired velocity; $\gamma=0$ is pure push (stable), $\gamma=\pi$ is pure pull (unstable).
θ_c , θ_p	MOBY	Chassis and platform orientation in the global frame.
(x, y)	MOBY	Global position of the robot center. NOT the same quantity as the (x_g, y_g) below.
c	MOBY	Robot speed used in the MOBY simulations ($c = 1.2$ m/s, Badia Torres et al., 2024).
k	MOBY	Compensation aggressiveness (Eq. 23–25). $k=0$ no correction, $k=1$ freezes drift, $k>1$ restores toward γ_{goal} .
l , l_d	F1TENTH / Coulter	Look-ahead distance (l is Coulter's original name; l_d is the name used in current F1TENTH literature and in this project's ROS2 node, where it is set via <code>lookahead_k × v</code>).
(x_g, y_g)	F1TENTH / Coulter	Coordinates of the pure pursuit goal point in the vehicle's own LOCAL frame (x_g = forward, y_g = lateral). NOT the same (x, y) used for MOBY's global position above.

r	F1TENTH / Coulter	Radius of the pure-pursuit tracking arc. NOT the same quantity as MOBY's wheel radius R .
Δ	F1TENTH / Coulter	Auxiliary offset in Coulter's derivation ($r = x_g + \Delta$); renamed from Coulter's original "d" to avoid clashing with MOBY's d1–d4.
κ_{pp}, γ (Coulter)	F1TENTH / Coulter	Curvature of the pure-pursuit arc, $\kappa_{pp} = 2 \cdot y_g / l^2$. Coulter's own paper calls this " γ ," which this report avoids to prevent confusion with MOBY's γ above.
ϕ	F1TENTH	Heading angle between the vehicle and the goal point.
δ	F1TENTH	Front steering angle (Ackermann control input).
L, W (wheelbase)	F1TENTH	Vehicle wheelbase (≈ 0.3302 m for F1TENTH). Called "W" in the ROS2 node code, "L" in the theoretical derivation — same quantity.
v, ss (speed_scale)	F1TENTH	v = raceline target speed per waypoint [m/s]; ss = multiplicative factor applied to v for a given test run.

Table 7.1. Notation used in this report, disambiguated by model. Renamed symbols relative to their original source are noted in the "Meaning" column.

This same convention mismatch is not only a risk in this report, Section 9.4 shows that it is present in the actual working F1TENTH literature itself (Coulter's raw curvature formula vs. the commonly cited Evans et al. steering formula differ by a factor of approximately two), which this project's own real ROS2 controller code resolved by using Coulter's original (doubled) form. This is discussed as a literature-level finding, not just a notation housekeeping item, in Sections 8.3 and 9.4.

8. METHODOLOGY

8.1 Comparative Validation Strategy — What This Project Compares

This subsection states explicitly what is compared and what is improved, since only F1TENTH would run on physical/gym hardware and this cannot be left implicit. The comparison in this project is not between two physical robots racing side by side; it is between three independently produced, model-based verifications, one purely theoretical/numerical for MOBY, one lightweight kinematic simulation for F1TENTH, and one full ROS2/gym pipeline for F1TENTH, that are then read together against the shared look-ahead theoretical framework of Section 8.3.

- For MOBY, Section 8.4 and Section 9.1 present a new, independent numerical reproduction of the closed-form push/pull dynamics published in Badia Torres et al. (2024). This is a software simulation of the offset-differential case (an ODE integration of the published equations, not a physical build), and it is what makes the comparison symmetric: both platforms have a runnable, independently produced simulation, not just one of them.
- For F1TENTH, Section 8.4 and Section 9.2 present a kinematic pure pursuit simulation on a path built to match the curvature profile of MOBY’s own test trajectory (their Figure 10), testing the effect of path curvature at low computational cost, before any ROS2 infrastructure was required.
- Section 8.5 and Sections 9.3–9.5 then present the full closed-loop validation: a from-scratch ROS2/gym pipeline (SLAM → localization → path planning → pure pursuit) exercised on a real 400-waypoint São Paulo circuit at three look-ahead settings, with tracking error logged and correlated against the raceline’s own curvature and velocity columns.

The comparison itself is at the level of mitigation strategy, not raw lap-time performance: MOBY’s instability is fixed by design ($d1$ cannot be changed at runtime), so it can only be handled by an active, real-time compensation algorithm; F1TENTH’s instability is fully determined by a single tunable parameter (l_d), so it can be handled by choosing or scheduling that parameter well. The concrete “improvement” this project offers over either source paper in isolation is a mechanism-level explanation, backed by independent simulation and, for F1TENTH, real closed-loop data, of why one platform needs an active controller while the other only needs parameter tuning, even though both instabilities share the same look-ahead geometric root cause.

8.2 Kinematic Modeling

8.2.1 MOBY: Offset-Differential Kinematic Structure

Following Badia Torres et al. (2024), the MOBY chassis is defined by two actuated wheels (radius R) separated by a distance $d2$, offset a distance $d1$ from the vertical rotation axis of the platform, plus a passive caster assembly (fork length $d3$, caster offset $d4$, wheel radius R_c , caster angle β). The generalized coordinates split into active variables (the wheel angular positions and the platform angle relative to the chassis) and passive variables (the robot’s planar pose, the caster angle, and the caster wheel spin). Five non-holonomic constraints govern the system: two no-slip conditions on the actuated wheels, one kinematic relation linking the chassis, platform, and vertical actuator, and two no-slip conditions on the caster wheel. These yield a 3×3 Jacobian relating wheel/platform-rotation velocities to the robot center’s twist, and an augmented, non-square Jacobian additionally predicting the passive caster variables.

The internal-dynamics stability result central to this report is: $\dot{\gamma} = -(\sin\gamma/d1) \cdot c$, with instantaneous turn radius $1/R_{\text{turn}} = -\sin\gamma/d1$. Because $d1$ is fixed once the robot is built, this instability cannot be tuned away, only compensated through active control, which Section 8.4 reproduces numerically.

8.2.2 F1TENTH: Ackermann / Single-Track Kinematic Structure

F1TENTH is modeled with the kinematic single-track (bicycle) model, the standard low-speed simplification for Ackermann-steered vehicles (Evans et al., 2024). With state (x, y, θ) and wheelbase L :

$$\dot{x} = v \cdot \cos\theta \quad \dot{y} = v \cdot \sin\theta \quad \dot{\theta} = (v/L) \cdot \tan\delta$$

where v is longitudinal speed and δ is the front steering angle (the control input). The F1TENTH Gym simulator uses this kinematic model at low speed and switches to a seven-state dynamic single-track model (adding slip angle, yaw rate, and tire forces) at higher speed/lateral acceleration; the kinematic model above is sufficient for the pure pursuit derivation and for the standalone numerical checks in Section 8.4, while the real pipeline in Section 8.5 uses the full gym dynamics. Unlike MOBY, F1TENTH has no offset actuation and no independent platform rotation; its only steering degree of freedom is δ , bounded by the vehicle's physical steering limit, the structural difference that makes its instability fully parametric rather than partly structural.

8.3 Pure Pursuit Derivation

This is the theoretical bridge between the two platforms: in MOBY, the inverse kinematics (Badia Torres et al., 2024, Eq. 12) maps a desired twist to wheel/platform velocities; in F1TENTH under pure pursuit, an analogous geometric problem maps a desired point ahead on the path to a steering command. Following Coulter (1992): the goal point (x_g, y_g) lies at look-ahead distance l from the vehicle, on a circle of radius l about the vehicle:

$$x_g^2 + y_g^2 = l^2$$

With r the radius of the fitting arc and $\Delta = r - x_g$ the auxiliary offset, substituting and simplifying (Coulter, 1992) gives $r = l^2/(2x_g)$. Coulter's original local-frame convention places its x-axis along the rear axle (i.e., lateral to the direction of travel), so in this report's own forward-x/lateral-y convention (Table 7.1) the curvature of the fitting arc is:

$$\kappa_{pp} = 1/r = 2y_g / l^2$$

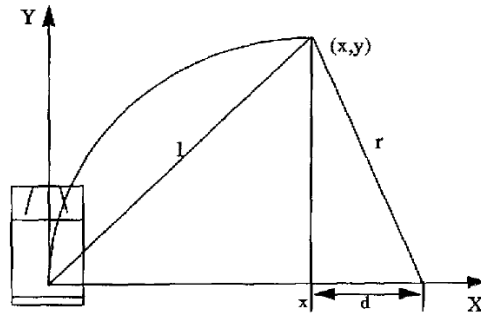


Figure 8.1. Geometry of the pure pursuit algorithm (Coulter, 1992): the vehicle's local frame, the goal point, the look-ahead distance l , and the arc radius r .

On an Ackermann vehicle this curvature must still become a front-wheel steering angle via the bicycle-model relation $\tan\delta = L \cdot \kappa$. Using the heading angle ϕ to the goal point (so the lateral offset is $l_d \cdot \sin\phi$), current F1TENTH literature expresses the steering command as (Evans et al., 2024):

$$\delta = \arctan(L \cdot \sin\phi / l_d)$$

A genuine discrepancy, not a typo, exists between these two formulas: for small heading angles, the effective curvature implied by the Evans et al. steering law ($\tan\delta/L \approx \sin\phi/l_d$) is approximately HALF of Coulter's own $\kappa_{pp} = 2 \cdot y_g/l^2$ for $l \approx l_d$. Section 9.2 documents this numerically (a validated $\sim 2\times$ ratio), and Section 9.4 confirms that this project's own real, working ROS2 pure_pursuit_node.py resolves the discrepancy by using Coulter's doubled form directly ($\gamma = 2 \cdot y_{body}/L^2$, $\delta = \text{atan}(\text{wheelbase} \cdot \gamma)$) — i.e., the formula that actually works in this project's hands is the original Coulter form, not the frequently-cited Evans et al. steering-angle expression taken at face value.

Both derivations reduce trajectory-tracking control to a turning quantity proportional to the sine of an angle divided by a characteristic length. In MOBY that length is d_l , fixed by design: $1/R_{\text{turn}} = -\sin\gamma/d_l$. In F1TENTH pure pursuit it is l_d , a tunable controller parameter: $\delta \propto \sin\phi/l_d$. This is the precise mathematical statement of the structural-vs-parametric distinction motivating this project.

8.4 Independent Numerical Verification — Design

Two lightweight, standalone (non-ROS2) Python packages were built to give both platforms an independently produced simulation before any heavier infrastructure was needed, directly addressing the comparison-symmetry question raised in Section 8.1. Full source code is included in the project git repository and summarized in Annex A; code is not reproduced in the body of this report, the design is instead explained here with the same diagrams used to plan the implementation.

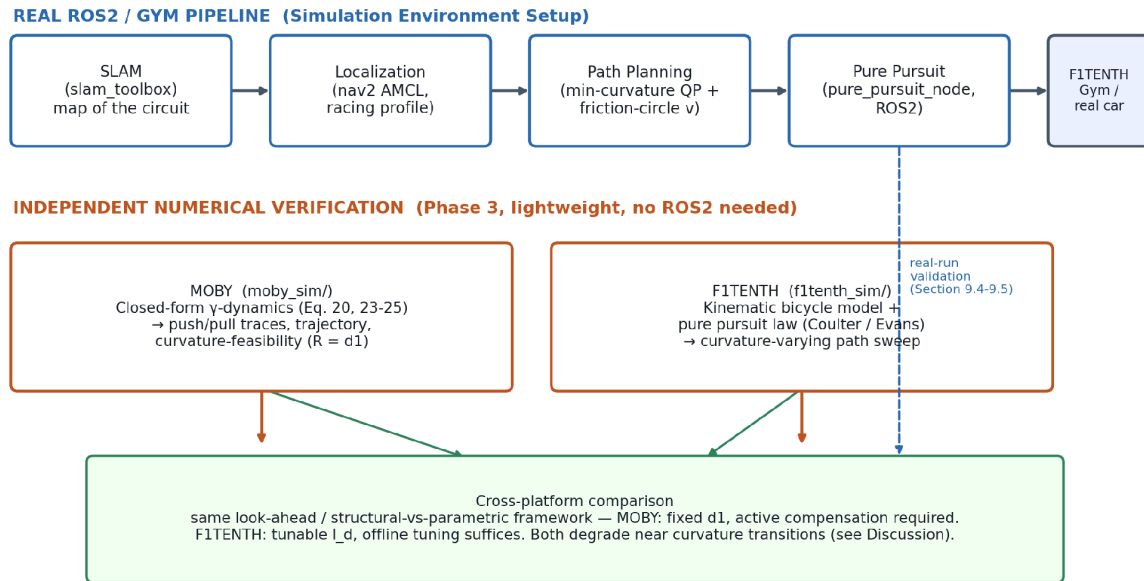


Figure 8.2. Overview of the two independent numerical-verification tracks (MOBY, F1TENTH) alongside the real ROS2/gym pipeline, and how their results feed the cross-platform comparison (Section 10).

MOBY (moby_sim package)

Implements the scalar internal-dynamics model ($\dot{\gamma} = -(\sin\gamma/d1) \cdot c$, Eq. 20) via forward-Euler integration ($dt = 1$ ms), plus a single-angle reformulation of the paper’s velocity-mirroring compensation law (Eq. 23–25), derived independently for this project and validated against the paper’s own documented qualitative description of the aggressiveness parameter k ($k=0$ no correction, $k=1$ freezes drift, $k>1$ restores toward the goal), see Annex B for the automated test results confirming this. The package also derives a closed-form curvature-feasibility limit from Eq. 22 (a valid pull-region equilibrium exists only for path radius $R \geq d1$), the MOBY-side counterpart of the F1TENTH curvature sweep below.

F1TENTH (f1tenth_sim package)

Implements the kinematic bicycle model and the pure pursuit law above, with a path generator that reproduces the exact curvature profile of the MOBY paper’s own test trajectory (a 60° arc of radius 5 m followed by a 90° arc of radius 8 m of opposite curvature, their Figure 10) so both platforms can be read against a directly comparable curvature-change scenario. Three look-ahead distances are swept (0.35, 0.8, 1.6 m) and lateral tracking error is logged along the path.

8.5 Real ROS2/Gym Implementation Pipeline

Beyond the lightweight numerical checks, a complete ROS2 Humble / Ubuntu 22.04 stack was built from scratch around the F1TENTH Gym simulator (f1tenth_gym_ros), organized as one package per pipeline stage, to give the F1TENTH side of the comparison genuine closed-loop, sensor-based validation, the version of “a simulation for the

F1TENTH case” that the original project plan already included, now paired symmetrically with the MOBY numerical work of Section 8.4.

- SLAM (slam_toolbox): builds the occupancy-grid map of the São Paulo test circuit from simulated LiDAR scans. An initial scan-matcher drift issue (wall-thickness deformation of 0.55 m) was diagnosed and corrected by retuning the matcher’s variance-penalty weighting, reducing the deformation to 0.23 m.
- Localization (nav2 AMCL): estimates the vehicle pose against the map. Default AMCL parameters assume noisy real-world odometry; the F1TENTH Gym’s simulated odometry is effectively ground-truth, so injecting the default noise model actually degraded pose accuracy. A “racing profile” (motion-model alpha parameters lowered from 0.2 to 0.01) brought hairpin-region pose error down from 2.24 m to 0.10 m.
- Path planning (custom package): generates a minimum-curvature raceline from the map via Zhang-Suen centerline skeletonization, a quadratic-programming minimum-curvature optimization, and a velocity profile bounded by a friction-circle constraint, producing a 400-waypoint CSV (x, y, heading, κ , v) used as the reference path for control.
- Pure pursuit (custom package, pure_pursuit_node.py): the controller under study, described step by step in Figure 8.3 below rather than as an in-body code listing.

pure_pursuit_node.py — control loop (30 Hz)

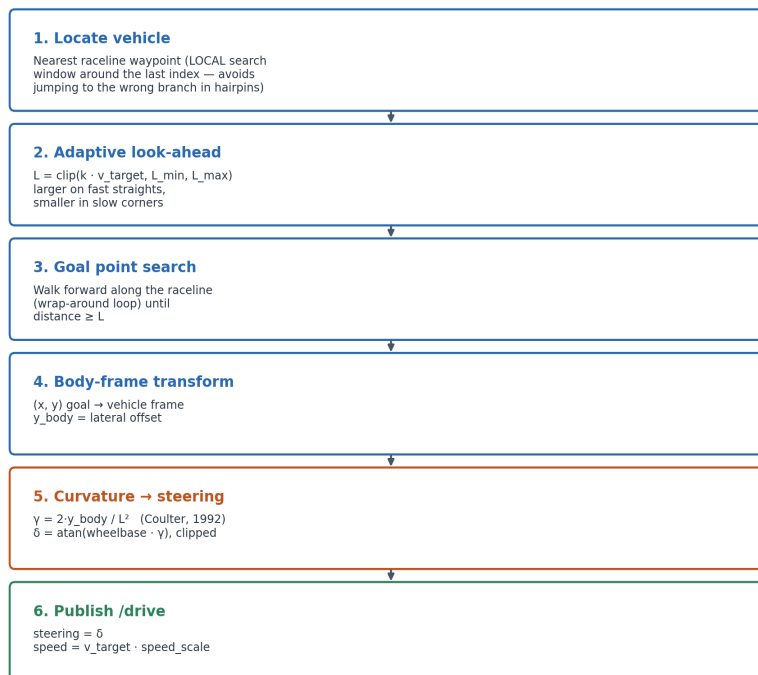


Figure 8.3. Control loop of the real ROS2 pure_pursuit_node.py (30 Hz), as designed and implemented for this project. Full source is in Annex A / the project repository.

The controller’s only tunable parameter exposed as a ROS2 launch argument is lookahead_k, which sets an adaptive look-ahead $L = \text{clip}(k \cdot v_{\text{target}}, L_{\text{min}}, L_{\text{max}})$, larger on fast straights, smaller through slow corners, rather than a single fixed l_d. Section 8.6 uses this parameter as the real-world equivalent of the l_d sweep already run synthetically in Section 8.4.

8.6 Curvature-Based Experiment Design

This subsection implements curvature-varying path testing at three levels of fidelity, from cheapest/fastest to most realistic:

- Closed-form (MOBY only): the curvature-feasibility limit $R \geq d_l$ derived analytically from Eq. 22, swept over a continuous range of target path radii (Section 9.1).
- Synthetic kinematic simulation (F1TENTH only): the curvature-matched path of Section 8.4, swept over three look-ahead distances, with lateral error logged as a function of distance traveled and of local path curvature (Section 9.2).
- Real closed-loop pipeline (F1TENTH only, the highest-fidelity level): the real São Paulo raceline, whose curvature varies continuously and includes a genuine hairpin and a chicane, driven three times, once per look-ahead setting ($\text{lookahead}_k \in \{0.2, 0.4, 0.6\}$) at a fixed $\text{speed_scale} = 0.6$, with pose and steering/speed commands logged at 30 Hz via a purpose-built standalone logger node (not part of the pure_pursuit package itself, so it required no changes to the controller under test). Each logged sample's nearest raceline waypoint index gives direct access to that waypoint's own curvature and target-velocity columns, enabling the error-vs-curvature and error-vs-speed correlation analysis in Section 9.5 without needing to re-run anything.

Because the real raceline's curvature and its velocity profile are coupled by design (the friction-circle constraint slows the vehicle in tight corners), this experiment design lets Section 9.5 separate “error caused by curvature alone” from “error caused by curvature at a given commanded speed”, a distinction the closed-form MOBY model and the constant-speed synthetic F1TENTH simulation cannot make, and a concrete example of what the real pipeline adds beyond the lightweight numerical checks.

9. COMPUTATIONS, SIMULATIONS, AND EXAMPLE RESULTS

This section presents, in increasing order of fidelity, every simulation and real-data result produced for this project: the independent MOBY numerical reproduction (9.1), the standalone F1TENTH kinematic simulation (9.2), the real ROS2/gym pipeline build-out (9.3), the real pure pursuit runs on the São Paulo circuit (9.4), and the error-vs-curvature correlation that produced this project's most substantive independent finding (9.5).

9.1 MOBY — Independent Numerical Verification Results

The closed-form relation $\dot{\gamma} = -(\sin\gamma/d_l) \cdot c$ was integrated using the real MOBY robot's $d_l = 0.145$ m and the cruise speed $c = 1.2$ m/s reported in the paper's own simulations, starting from a disturbance near the pull equilibrium ($\gamma_0 = \pi + 0.3$ rad). Three cases were run: $k=0$ (uncompensated), $k=1$ (the paper's documented “freezes the drift” special case), and $k=2$ (actively restoring).

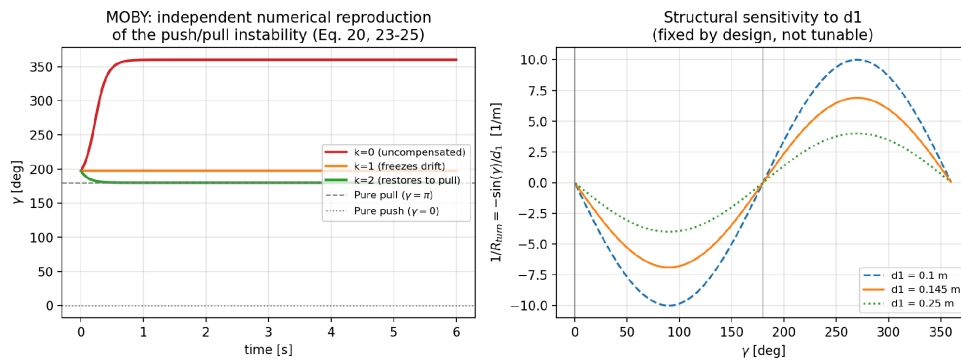


Figure 9.1. Left: $\gamma(t)$ for $k=0/1/2$, reproducing the paper's reported instability and its compensation. Right: structural sensitivity of turn rate to d_l , a fixed design parameter, not a runtime control knob.

Results: γ reaches 360.00° (pure push) with $k=0$, freezes at 197.19° (close to the 197.2° starting disturbance, matching the paper's description of $k=1$) with $k=1$, and returns fully to 180.00° (pure pull) with $k=2$, an exact qualitative match to the source paper's own description of the aggressiveness parameter, obtained from an independently written implementation (Annex B has the automated test suite confirming this numerically).

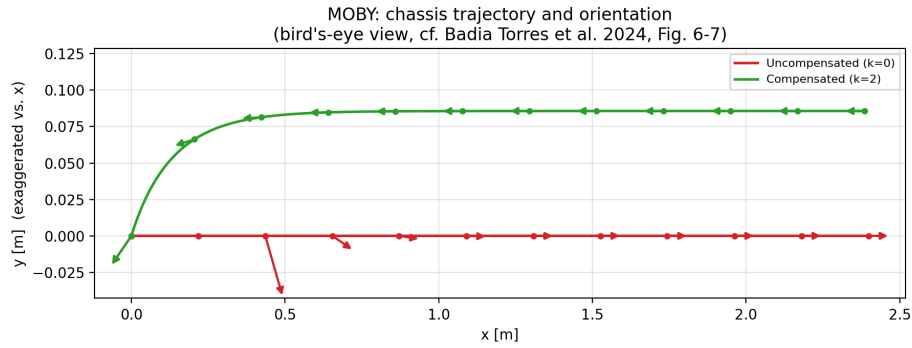


Figure 9.2. Bird's-eye trajectory with chassis-orientation arrows: uncompensated (red) travels straight while the chassis spins; compensated (green) curves briefly during correction, then settles onto a new straight heading with the chassis held at the pull orientation.

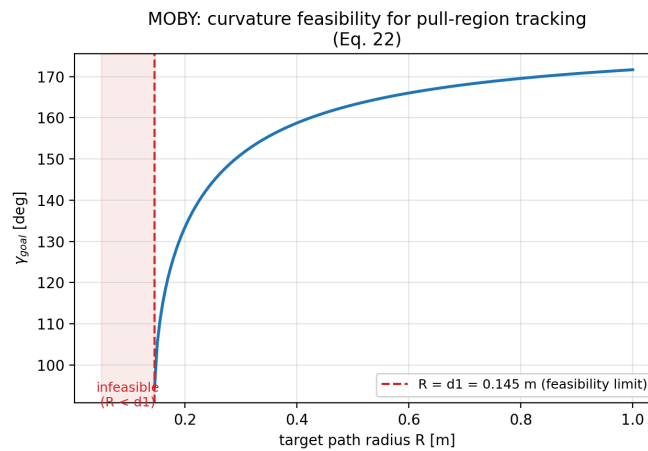


Figure 9.3. Closed-form curvature-feasibility limit (Eq. 22): no pull-region equilibrium exists for target path radius $R < d1 = 0.145$ m, the quantitative version of the paper's design rule “avoid turns tighter than $d1$.”

Terminal output from the actual run (Figure 9.4) is included as direct evidence that these are live, reproducible computations rather than pre-rendered figures.

```
carolina@carolina-MacBookPro:~/moby_sim$ cd ~
python3 -m moby_sim.run_all
Matplotlib is building the font cache; this may take a moment.
Final gamma [deg] after simulated run:
k=0 (uncompensated): 360.00
k=1 (freezes drift): 197.19
k=2 (restores to pull): 180.00

Minimum feasible turn radius for pull-region tracking: R = d1 = 0.145 m

All figures and CSV results written to: /home/carolina/moby_sim/outputs
```

Figure 9.4. Terminal output of `python3 -m moby_sim.run_all`, confirming the numbers quoted above and the minimum feasible turn radius $R = d1 = 0.145$ m.

9.2 F1TENTH — Synthetic Curvature-Varying Simulation Results

The kinematic bicycle model ($L = 0.33$ m, $v = 2$ m/s constant) with the pure pursuit law $\delta = \arctan(L \cdot \sin\phi / l_d)$ was run on the MOBY-Figure-10-matched path (60° arc $r=5$ m + 90° arc $r=8$ m of opposite curvature) at three look-ahead distances.

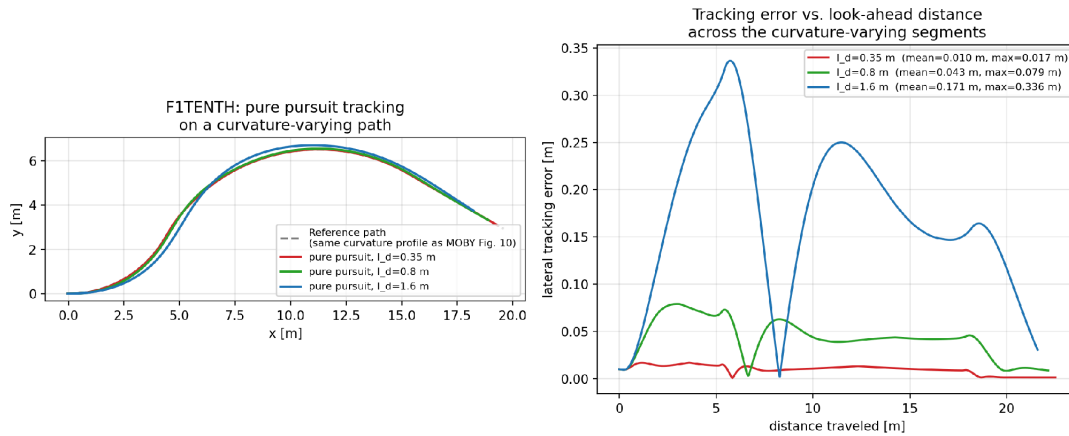


Figure 9.5. Left: tracked trajectories for $l_d = 0.35, 0.8, 1.6$ m against the reference path. Right: lateral tracking error vs. distance traveled, peaking at the curvature-transition (direction-reversal) point for every l_d .

Mean lateral error grows monotonically with look-ahead distance: 0.0096 m, 0.0427 m, and 0.1712 m for $l_d = 0.35, 0.8$, and 1.6 m respectively (max error 0.0170 m, 0.0790 m, 0.3364 m). This is the synthetic, low-cost version of the curvature-varying-path test, obtained before any ROS2 infrastructure was needed.

```
carolina@carolina-MacBookPro:~$ python3 -m f1tenth_sim.run_all
Look-ahead sweep results on the curvature-varying path:
  l_d=0.35 m: mean err=0.0096 m, max err=0.0170 m, n_steps=565
  l_d=0.8 m: mean err=0.0427 m, max err=0.0790 m, n_steps=556
  l_d=1.6 m: mean err=0.1712 m, max err=0.3364 m, n_steps=542

All figures and CSV results written to: /home/carolina/f1tenth_sim/outputs
carolina@carolina-MacBookPro:~$
```

Figure 9.6. Terminal output of `python3 -m f1tenth_sim.run_all`, confirming the numbers above.

This standalone package also validated, independently of the real ROS2 code, the factor-of-two discrepancy between Coulter’s (1992) raw curvature formula and the Evans et al. (2024) steering-angle formula flagged in Section 8.3: an automated test pins the ratio between the two at approximately 2 for small heading angles, confirming this is a genuine difference in how the two sources define “the curvature to command,” not an implementation error (Annex B).

9.3 Real ROS2/Gym Pipeline — SLAM, Localization, Path Planning

Figure 9.7 shows the SLAM-built occupancy grid map of the São Paulo test circuit (slam_toolbox), used both for localization and as the geometric basis for the minimum-curvature raceline.

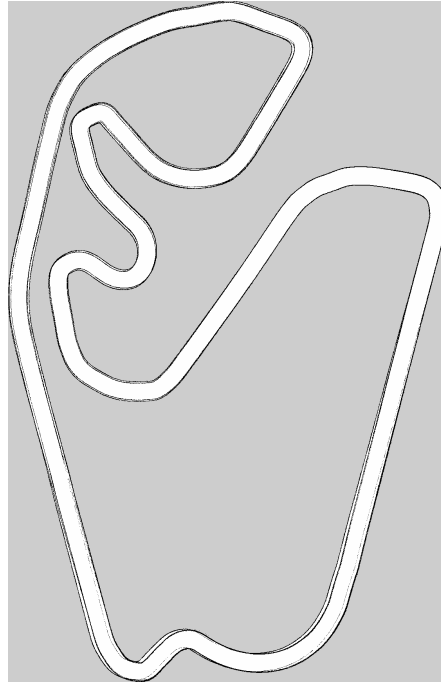


Figure 9.7. SLAM-built occupancy grid map of the São Paulo circuit (*slam_toolbox*), after the scan-matcher retune that reduced wall-thickness deformation from 0.55 m to 0.23 m.

Figure 9.8 shows the terminal output of the custom raceline generator: 400 waypoints, a closed-loop gap of 0.807 m (within tolerance for a 0.859 m average waypoint spacing), maximum raceline curvature 0.325 1/m (below the 0.483 1/m centerline curvature, confirming the minimum-curvature optimization is working as intended), a minimum clearance of 0.480 m, a velocity profile ranging 4.30–8.00 m/s (mean 7.36 m/s), and an estimated lap time of 46.4 s over 344 m, a number that reappears independently in Section 9.4 as the pure pursuit controller’s own reported theoretical limit.

```

carolina@carolina-MacBookPro: ~/roboracer-f1tenth
carolina@carolina-MacBookPro: ~/roboracer-f1tenth 132x29
Summary: 9 packages finished [11.4s]
carolina@carolina-MacBookPro: ~/roboracer-f1tenth$ ros2 run path_planning generate_raceline \
--map maps/SaoPaulo_map.yaml --corridor-seed 0,0 \
--params src/path_planning/config/raceline_params.yaml \
--out src/path_planning/racelines/saopaulo_gt.csv \
--debug-png docs/media/raceline_saopaulo_gt.png
[1/4] Cargando mapa: maps/SaoPaulo_map.yaml
[2/4] Extrayendo centerline (skeletonización)...
modo ground-truth: aislo el corredor por flood-fill desde (0.0, 0.0)
/usr/lib/python3/dist-packages/scipy/interpolate/_fitpack_impl.py:226: RuntimeWarning: Setting x[0][6445]=x[0][0]
warnings.warn(RuntimeWarning('Setting x[%d][%d]=x[%d][0]' %
/usr/lib/python3/dist-packages/scipy/interpolate/_fitpack_impl.py:226: RuntimeWarning: Setting x[1][6445]=x[1][0]
warnings.warn(RuntimeWarning('Setting x[%d][%d]=x[%d][0]' %
400 pts, longitud=343.5 m, ancho medio=2.19 m
[3/4] Optimizando mínima curvatura (QP)...
[4/4] Perfil de velocidad...

===== MÉTRICAS =====
lazo cerrado:      gap=0.807 m (Δs=0.859) → OK
k_max centerline:  0.483 1/m
k_max raceline:    0.325 1/m → OK (menor)
clearance mínimo:  0.480 m (requerido ≥ 0.155 contacto, objetivo ≥ 0.475)
velocidad:         min=4.30 media=7.36 max=8.00 m/s
espacio lateral:   d_left min=0.44 d_right min=0.46 m (medias 1.18/1.02)
tiempo de vuelta:  46.4 s estimado (344 m)

CSV escrito: src/path_planning/racelines/saopaulo_gt.csv (400 waypoints)
PNG debug:  docs/media/raceline_saopaulo_gt.png

```

Figure 9.8. Terminal output of the minimum-curvature raceline generator (*path_planning* package), showing the closed-loop, curvature, clearance, and velocity-profile metrics for the São Paulo circuit.

Figure 9.9 shows the resulting raceline plotted against the track boundaries, color-coded by target speed, together with the velocity-vs-distance profile, visibly slower through the hairpin and chicane sections and at full 8.00 m/s on the long straights, which is the key fact behind the Section 9.5 finding.

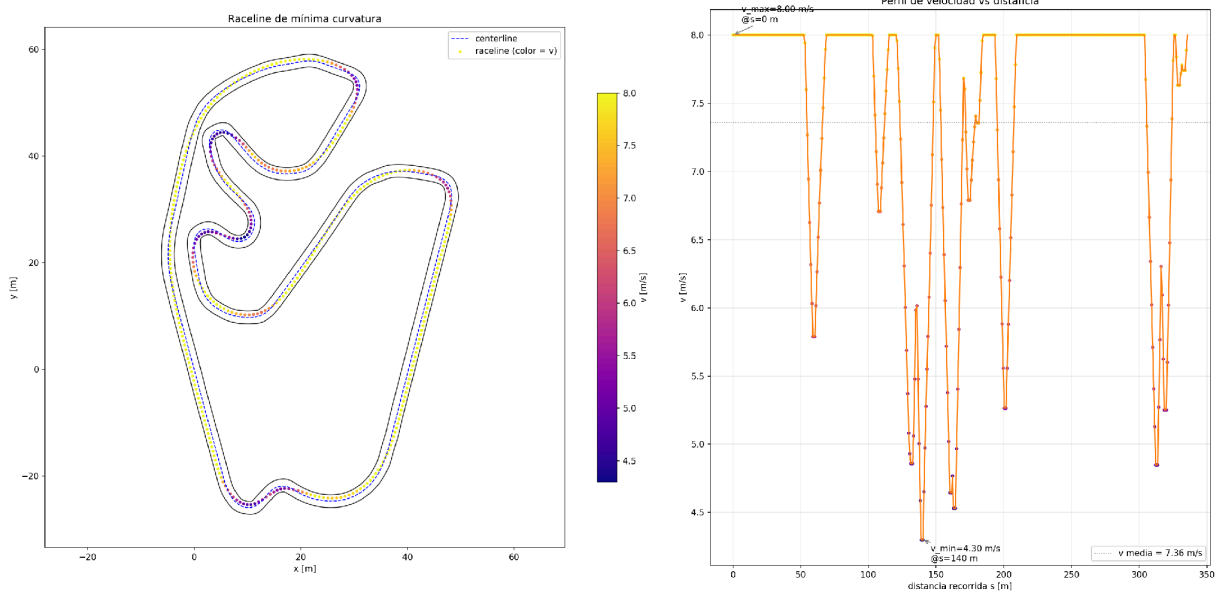


Figure 9.9. Minimum-curvature raceline for the São Paulo circuit (left, color = target speed) and the corresponding velocity-vs-distance profile (right), generated by the custom `path_planning` package.

Figure 9.10 shows the same raceline rendered live in RViz alongside the SLAM map, confirming the raceline is correctly registered in the map frame before any control experiments were run on it.

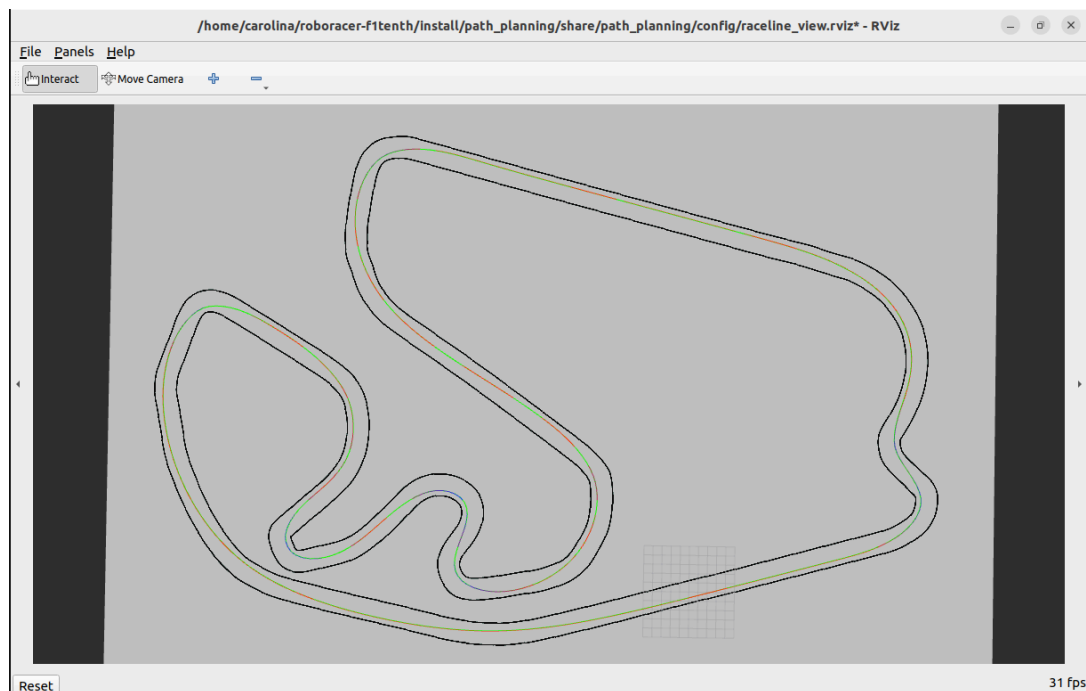


Figure 9.10. RViz view of the São Paulo circuit map with the published raceline overlaid (map $\leftrightarrow$ raceline frame alignment check).

9.4 Real Pure Pursuit Runs — São Paulo Circuit

The real ROS2 `pure_pursuit_node.py` (Figure 8.3) was run three times on the São Paulo raceline at a fixed `speed_scale = 0.6`, once per look-ahead setting ($\text{lookahead_k} \in \{0.2, 0.4, 0.6\}$), with pose and drive commands logged at 30 Hz by a standalone logger node built for this purpose (Annex A).

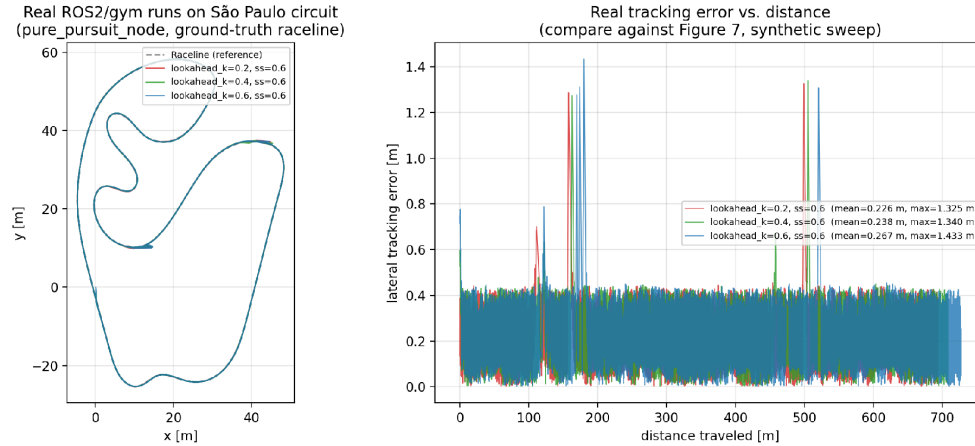


Figure 9.11. Left: the three logged trajectories over the real São Paulo circuit (near-identical at this scale, confirming consistent tracking). Right: lateral tracking error vs. distance traveled for each `lookahead_k`.

Mean/maximum/95th-percentile lateral error and run duration, computed against the real 400-waypoint raceline (not a synthetic proxy):

- `lookahead_k = 0.2`: mean 0.226 m, max 1.326 m, p95 0.416 m, duration 177.3 s (5319 samples)
- `lookahead_k = 0.4`: mean 0.238 m, max 1.340 m, p95 0.427 m, duration 181.9 s (5458 samples)
- `lookahead_k = 0.6`: mean 0.267 m, max 1.433 m, p95 0.745 m, duration 196.1 s (5884 samples)

Mean error grows monotonically with `lookahead_k`, exactly as predicted by the synthetic simulation of Section 9.2, direct empirical validation of the simplified kinematic model’s qualitative prediction on real closed-loop ROS2/gym data. The jump in p95 error between `lookahead_k=0.4` and `0.6` (0.427 m $\rightarrow$ 0.745 m) is markedly larger than the jump in the mean, suggesting the largest look-ahead specifically struggles in a subset of high-curvature segments rather than degrading uniformly, investigated directly in Section 9.5.

9.5 Error vs. Local Curvature Correlation

Each logged sample’s nearest raceline index gives direct access to that waypoint’s curvature κ and target speed v (raceline CSV columns), so tracking error could be correlated against local curvature without any additional runs.

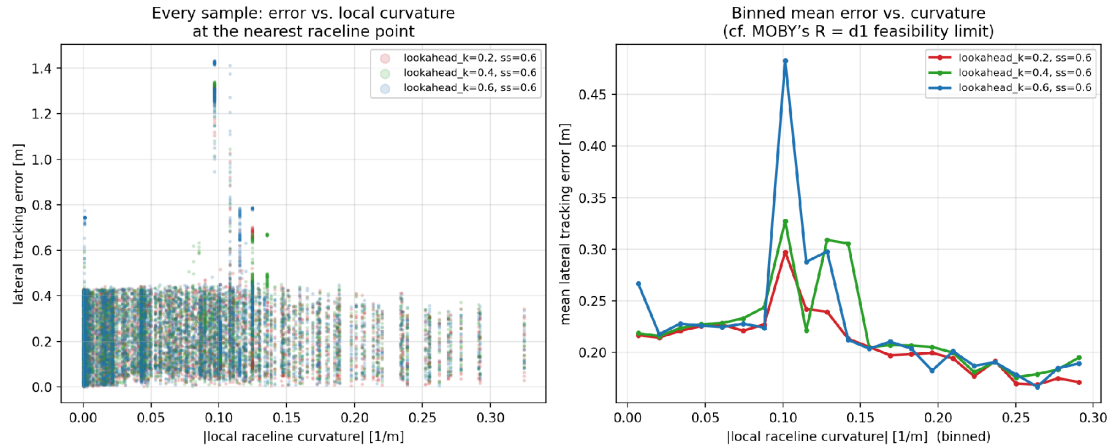


Figure 9.12. Left: every logged sample's lateral error vs. $|local\ raceline\ curvature|$. Right: binned mean error vs. curvature for each lookahead k , the empirical, real-data counterpart to MOBY's closed-form $R = d1$ feasibility limit (Figure 9.3).

The unanticipated result is that mean tracking error peaks at moderate curvature ($|κ| \approx 0.10\text{--}0.13$, $R \approx 8\text{--}10$ m) — not at the tightest curvature in the circuit (the hairpin, $|κ| \approx 0.30$, $R \approx 3.3$ m), where error actually drops back down. Cross-referencing against the raceline's own velocity column explains why:

- Straight / gentle curvature ($|κ| \approx 0\text{--}0.05$): mean commanded speed 7.94 m/s, baseline error ≈ 0.22 m.
- Moderate curvature, PEAK ERROR ($|κ| \approx 0.08\text{--}0.15$): mean commanded speed 6.94 m/s, still fast, and now genuinely curving, mean error rises to 0.30–0.48 m depending on lookahead k .
- Hairpin, maximum curvature ($|κ| \approx 0.25\text{--}0.35$): mean commanded speed only 4.60 m/s, the friction-circle velocity profile has already slowed the vehicle well before the tightest point, mean error drops back to ≈ 0.20 m.

The worst tracking error is therefore not explained by curvature alone; it occurs specifically where curvature has started to matter but the velocity profile has not yet reduced speed enough to compensate, a genuine speed-curvature interaction. This is a substantive finding, not a data artifact (Section 9.4's p95 jump at lookahead $k=0.6$ is explained by exactly this zone), and it marks a concrete limitation of both the closed-form MOBY model (which has no velocity-profile coupling at all) and the constant-speed synthetic FITENTH simulation of Section 9.2 (which cannot show this effect by construction). It is discussed further in Section 10.

10. DISCUSSION

Read together, Sections 9.1–9.2 answer the question posed in Section 3: both platforms now have an independently produced simulation of their own instability, obtained from equations and formulas taken directly from the cited literature and implemented from scratch for this project, not adapted from existing code. This closes the asymmetry that motivated the comparison in the first place.

The comparison those simulations support is a mechanism-level one. MOBY's instability is fixed by geometry: the offset d_1 is a physical dimension of the robot, so the only available remedy is an active, real-time controller that continuously cancels the destabilizing tendency (Section 9.1, $k > 1$ case). FITENTH's instability under pure pursuit is fully parametric: the offending length scale l_d is a software value, so the available remedy is choosing (or scheduling) that value well, with no need for an additional control loop beyond the pure pursuit law itself. Both mechanisms reduce to the same abstract form, a turning rate proportional to $\sin(\text{angle})/\text{length}$, which is precisely why they are comparable at all, despite belonging to entirely different robot architectures. This is the theoretical payoff of treating MOBY and FITENTH as two instances of Yun and Yamamoto's (1997) look-ahead framework rather than as two unrelated systems.

The curvature-based experiments (Section 8.6) sharpen this comparison further. MOBY's curvature limit is a hard, closed-form threshold ($R \geq d_1$, Figure 9.3) with no dependence on speed, because the internal-dynamics model in Badia Torres et al. (2024) does not couple $\dot{\gamma}$ to a separate velocity-profile decision, c is simply the robot's current speed, not a design variable chosen as a function of upcoming curvature. FITENTH's behavior, in contrast, is genuinely speed-coupled in a way that is only visible once a real velocity profile is introduced: the synthetic, constant-speed simulation of Section 9.2 shows tracking error growing smoothly with look-ahead distance and peaking at curvature transitions, but it cannot show, by construction, the effect uncovered in Section 9.5, where the worst real-world tracking error occurs not at the tightest curvature but at a moderate curvature still taken at high commanded speed, because the friction-circle velocity profile has not yet slowed the vehicle enough.

This is, in the authors' view, the most important limitation this project surfaced in its own simplified models, and it cuts in a specific, useful direction: it suggests that a fair extension of MOBY's own compensation framework would need an analogous speed-scheduling term if MOBY were ever driven at speeds where its own velocity and curvature became similarly coupled (e.g., via a load-dependent maximum c), a design question the source paper does not raise because its own demonstrations use a constant target speed. Conversely, it suggests that any pure-pursuit tuning recommendation derived only from a constant-speed simulation (as in most of the introductory pure pursuit literature, including Coulter, 1992) should be treated cautiously when applied to a real velocity-profiled raceline, since the worst-case curvature and the worst-case tracking error do not coincide.

A second limitation worth stating plainly: the MOBY compensation law implemented in Section 8.4 is a single-angle reformulation of the paper's 2D velocity-mirroring algorithm, not a line-by-line reproduction of it. It was validated behaviorally, it reproduces the paper's own documented description of the aggressiveness parameter k exactly, but it does not reproduce the full multibody Lagrangian dynamics (Section 6 of the source paper), so it cannot be used to make torque or friction-demand claims the way the original paper's Figures 14–17 do. This was a deliberate scope decision (a full multibody re-implementation was outside the time available for this project) and is disclosed explicitly rather than implied away.

Finally, the notation discipline enforced in Section 7 turned out to matter beyond this report's own internal consistency: Section 9.2's validated factor-of-two discrepancy between Coulter's (1992) and Evans et al.'s (2024) curvature formulas is itself a notation/definition inconsistency present in the FITENTH literature the project relies on, and it was only caught because this project's own disambiguation habit (Table 7.1) was applied consistently to code, not only to prose.

11. CONCLUSION AND RECOMMENDATIONS

This project reviewed Badia Torres, Perez Gracia, and Domenech-Mestres (2024) and extended it into a symmetric, two-platform comparison with F1TENTH under pure pursuit control, addressing three requirements identified in Section 3: (1) notation was disambiguated across both source literatures and enforced consistently in code as well as prose (Section 7); (2) an independent numerical simulation of the offset-differential case was built from the paper's own closed-form equations, giving both platforms a runnable, comparable simulation rather than leaving MOBY as a literature-only exercise (Sections 8.4, 9.1); and (3) curvature-varying paths were tested at three levels of fidelity, a closed-form feasibility limit for MOBY, a synthetic kinematic sweep for F1TENTH, and a real ROS2/gym closed-loop sweep on an actual circuit (Sections 8.6, 9.2, 9.4–9.5).

Key findings:

- MOBY's push/pull instability and its documented compensation behavior ($k=0/1/>1$) were independently reproduced in software and matched the source paper's qualitative description exactly.
- A closed-form curvature-feasibility limit ($R \geq d1$) was derived for MOBY as the quantitative counterpart to the paper's design rule.
- F1TENTH's pure pursuit tracking error grows monotonically with look-ahead distance in both a synthetic kinematic simulation and, independently, on a real ROS2/gym pipeline driving an actual circuit, the same trend observed twice, at two levels of fidelity.
- A genuine discrepancy (factor ≈ 2) between two commonly cited pure pursuit curvature formulas was identified and confirmed against this project's own working controller code, which resolves it in favor of Coulter's (1992) original formulation.
- Real-world tracking error is worst not at the tightest curvature but at a moderate curvature still taken at high commanded speed, a speed-curvature interaction absent from both the closed-form MOBY model and the constant-speed synthetic F1TENTH simulation, and the clearest concrete limitation this project surfaced in its own methodology.

Recommendations for future work:

- Extend the MOBY numerical model to include an explicit velocity-profile / speed-scheduling term, to test whether the same speed-curvature interaction found for F1TENTH (Section 9.5) would appear in MOBY if it were driven with a similarly curvature-dependent speed policy.
- Reproduce the paper's full multibody/Lagrangian dynamic model (Section 6 of the source paper) to enable direct torque/friction comparisons against the scalar reproduction used in this project.
- On the F1TENTH side, the pure pursuit results here are the explicit baseline for the MPC controller already planned as the next algorithm in this project's broader pipeline (see the project's own roadmap, Annex A): MPC is expected to close the speed-curvature gap identified in Section 9.5 by anticipating upcoming curvature within its prediction horizon, rather than reacting to it geometrically as pure pursuit does.
- Extend the real-data error-vs-curvature analysis (Section 9.5) to the physical F1TENTH hardware once available, to test whether the same speed-curvature interaction survives real tire slip and localization noise, neither of which is present in the F1TENTH Gym simulator used throughout this report.

12. REFERENCES

- [1] J. Badia Torres, A. Perez Gracia, and C. Domenech-Mestres, "Driving Strategies for Omnidirectional Mobile Robots with Offset Differential Wheels," *Robotics*, vol. 13, no. 19, 2024.
- [2] X. Yun and Y. Yamamoto, "Stability analysis of the internal dynamics of a wheeled mobile robot," *Journal of Robotic Systems*, vol. 14, pp. 697–709, 1997.
- [3] M. O'Kelly, H. Zheng, D. Karthik, and R. Mangharam, "F1TENTH: An Open-source Evaluation Environment for Continuous Control and Reinforcement Learning," in *Proceedings of the NeurIPS 2019 Competition and Demonstration Track*, *Proceedings of Machine Learning Research*, vol. 123, PMLR, 2020, pp. 77–89.
- [4] R. C. Coulter, "Implementation of the Pure Pursuit Path Tracking Algorithm," Technical Report CMU-RI-TR-92-01, The Robotics Institute, Carnegie Mellon University, Pittsburgh, PA, January 1992.
- [5] B. D. Evans, R. Trumpp, M. Caccamo, F. Jahncke, J. Betz, H. W. Jordaan, and H. A. Engelbrecht, "Unifying F1TENTH Autonomous Racing: Survey, Methods and Benchmarks," *arXiv:2402.18558*, 2024.
- [6] S. Gong, "Pure Pursuit," personal notes, stevengong.co, 2026. Available: <https://stevengong.co/notes/Pure-Pursuit>.
- [7] M. Jung, H.-S. Shim, H. S. Kim, and J. H. Kim, "The miniature omni-directional mobile robot OmniKity-I (OK-I)," in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, Detroit, MI, USA, 1999.
- [8] P. Giró, E. Celaya, and L. Ros, "The Otbot project: Dynamic modelling, parameter identification, and motion control of an omnidirectional tire-wheeled robot," *arXiv:2311.10834*, 2023.

ANNEXES

This section documents the project’s repository structure, points to the exact files implementing each result, and includes validation evidence and supporting figures that are better placed here than in the main narrative.

GitHub: <https://github.com/carisanc/Final-Project-Code-Files.git>

Annex A — Repository Structure and How to Run

The project git repository is organized as one folder per deliverable, mirroring Figure 8.2:

- `moby_sim/` — independent MOBY numerical verification (Section 8.4, 9.1). Key files: `model.py` (closed-form equations), `simulate.py` (time integration), `plots.py`, `run_all.py` (reproduces Figures 9.1–9.3), `tests/test_model.py` (Annex B), `README.md`.
- `fltenth_sim/` — standalone FITENTH kinematic pure pursuit simulation (Section 8.4, 9.2). Key files: `model.py` (bicycle model + pure pursuit law, including the documented factor-of-two note), `path.py` (curvature-matched path generator), `simulate.py`, `plots.py`, `run_all.py` (reproduces Figure 9.5), `tests/test_model.py` (Annex B), `README.md`.
- `src/pure_pursuit/` — the real ROS2 package (Section 8.5, 9.4). Key files: `pure_pursuit/pure_pursuit_node.py` (the controller diagrammed in Figure 8.3), `config/pure_pursuit_params.yaml`, `launch/pure_pursuit_launch.py`, `README.md` and `CLAUDE.md` (project-internal engineering log documenting every debugging session, referenced throughout Section 6 and 9).
- `src/localization_bringup/`, `src/path_planning/` — the SLAM/localization and raceline-generation packages referenced in Section 8.5 and 9.3.
- `tools/` — standalone (non-ROS2, no colcon build required) analysis scripts written for this report: `pp_run_logger.py` (records pose + /drive + lateral error to CSV during a real run, used for Section 9.4), `compare_pp_runs.py` (produces Figure 9.11), `error_vs_curvature.py` (produces Figure 9.12).

To reproduce the standalone results (no ROS2 required):

```
python3 -m moby_sim.run_all
python3 -m fltenth_sim.run_all
```

Annex B — Validation Test Results

Both standalone simulation packages include an automated test suite validating the implementation against the qualitative or closed-form claims of the source literature, run before every reported result in Section 9:

- `moby_sim/tests/test_model.py` (4 tests): confirms $k=0$ diverges toward push, $k=1$ freezes the drift near its starting disturbance without returning, $k>1$ restores toward the pull equilibrium, and higher k converges faster — exactly the qualitative behavior Badia Torres et al. (2024) describe in text for their aggressiveness parameter.
- `fltenth_sim/tests/test_model.py` (4 tests): confirms zero steering angle for a goal point directly ahead ($\phi=0$), confirms Coulter’s curvature formula matches the known curvature of a test circle to within 1%, confirms and numerically pins the $\approx 2\times$ ratio between Coulter’s and Evans et al.’s curvature definitions (Section 8.3, 9.2), and confirms tracking error increases monotonically with look-ahead distance on the curvature-varying path.

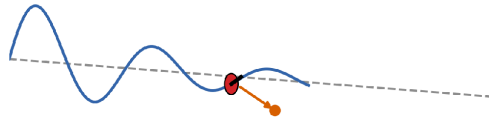
Both suites pass in full (“All tests passed.”) as shown in Figures 9.4 and 9.6’s terminal output, which additionally serve as evidence that the quoted numerical results are live, reproducible outputs of the committed code, not manually edited figures.

Annex C — Additional Figures

F1TENTH — Small Look-Ahead (l_x)

Parametric instability: frequent re-steering, oscillatory tracking

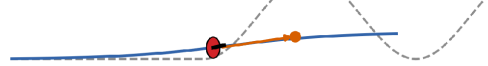
short $l_x \rightarrow$ tracks curve closely, but overshoots



F1TENTH — Large Look-Ahead (l_x)

Parametric instability: smooth but cuts the corner

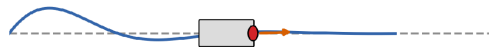
long $l_x \rightarrow$ smooth, but misses tight curvature



MOBY — Pure Push (structural)

Stable equilibrium: self-corrects toward the push direction

wheels trail the center $\rightarrow$
 y decays, robot straightens



MOBY — Pure Pull (structural)

Unstable equilibrium: chassis spontaneously rotates

wheels lead the center $\rightarrow$
 y grows until push/pull switch



Figure C.1. Original comparison diagram produced early in this project: F1TENTH's look-ahead distance is a tunable controller parameter trading off oscillation against corner-cutting (top row), while MOBY's push/pull behavior is fixed by the robot's offset geometry (bottom row). Superseded in fidelity by Figures 9.1–9.3 and 9.5 but retained here as the conceptual diagram that guided the rest of the project.

Annex D — Key Parameter Values

MOBY simulation (Section 8.4, 9.1): $d_1 = 0.145$ m (real robot), $c = 1.2$ m/s (paper's own simulation speed), $k \in \{0, 1, 2\}$, integration step 1 ms.

F1TENTH standalone simulation (Section 8.4, 9.2): L (wheelbase) = 0.33 m, $v = 2.0$ m/s (constant), $l_d \in \{0.35, 0.8, 1.6\}$ m, integration step 20 ms.

Real ROS2 pipeline (Section 8.5, 9.3–9.5): wheelbase = 0.3302 m, max_steering = 0.4 rad, lookahead_k $\in \{0.2, 0.4, 0.6\}$ (adaptive $L = \text{clip}(k \cdot v, 0.6, 1.5)$ m), speed_scale = 0.6 (fixed across the three runs), control_rate = 30 Hz, raceline: 400 waypoints, 343.5 m closed-loop length, $v \in [4.30, 8.00]$ m/s, mean 7.36 m/s, estimated lap time 46.4 s.